

NexusAI

PLATFORM

Complete Technical Architecture &
Implementation Blueprint

GenAI | RAG | Agentic Loops | ML Attrition | Designathon Edition

Auth Gate

TicketConsole

DocuMind RAG

SentinelHR

AgentFlow

Table of Contents

- 1. Platform Overview & Value Proposition
- 2. High-Level System Architecture
- 3. Technology Stack
- 4. Database Design — Full Schema
 - 4.1 Users & Sessions
 - 4.2 Tickets & Resolutions
 - 4.3 Knowledge Base (RAG)
 - 4.4 Employee & HR Data
 - 4.5 Agent Sessions
- 5. Module 1 — Auth Gate
- 6. Module 2 — TicketConsole
- 7. Module 3 — DocuMind RAG
- 8. Module 4 — SentinelHR
- 9. Module 5 — AgentFlow
- 10. API Endpoints Reference
- 11. Frontend Structure & UX Flow
- 12. Deployment Guide
- 13. Adaptability for Real Companies

1. Platform Overview & Value Proposition

NexusAI is an **AI-powered enterprise operations platform** that consolidates IT support ticketing, internal knowledge management, HR attrition intelligence, and autonomous web research into a single unified dashboard. Every module leverages a distinct AI technique — RAG for grounded Q&A;, supervised ML for predictive analytics, and ReAct agentic loops for dynamic research — making it a showcase of modern GenAI capabilities in a real-world enterprise context.

Capability	AI Technique	Business Value
IT ticket triage	VADER + BERT + RAG	Reduce operator workload via auto-resolution
Internal doc Q&A;	Embedding + RAG (Gemini)	Instant policy/knowledge retrieval without searching
HR attrition predict	XGBoost + SHAP	Proactive retention before talent leaves
Web research agent	ReAct + Gemini + Search	Grounded, cited answers — no hallucinations
Role-based access	JWT + RBAC	Secure, adaptable to any org hierarchy

2. High-Level System Architecture

The platform follows a clean **three-tier architecture**: a React SPA client, a Node.js + Express API server handling all AI orchestration, and a persistence layer combining a relational DB (PostgreSQL / SQLite) with a vector store (ChromaDB) for embedding-based retrieval.

Layer	Technology	Responsibility
Client	React (Vite) + Tailwind CSS + Lucide	SPA with role-aware routing, streaming AI responses, Chart.js visualisations
API Gateway	Node.js + Express.js	JWT auth middleware, request routing, AI orchestration, response streaming
AI Engine	Google Gemini 1.5 Flash + VADER + BERT rules	NLP scoring, embedding generation, LLM inference, ReAct loop execution
Relational DB	PostgreSQL (prod) / SQLite (dev)	Users, tickets, resolutions, employee records, session logs
Vector Store	ChromaDB (local) or pgvector	Document chunk embeddings for RAG semantic search
ML Runtime	Python (Flask micro-service) / joblib	XGBoost model inference + SHAP explanation generation

Request lifecycle — top to bottom:

- Browser sends HTTPS request with JWT in Authorization header.
- Express middleware verifies JWT, extracts { `userId`, `role` }, attaches to `req.user`.
- Role-scope guard checks if the requested route is allowed for that role.
- Route handler calls the relevant AI pipeline (RAG / ML / Agent).
- Response is streamed back via Server-Sent Events (SSE) for LLM outputs, or as plain JSON for structured data.
- Frontend renders result — streaming text, chart data, or ticket queue update.

3. Technology Stack

Area	Package / Tool	Version	Why
Frontend framework	React + Vite	18 / 5	Fast HMR, small bundle
Frontend styling	Tailwind CSS	3.x	Utility-first, rapid prototyping
Frontend icons	Lucide React	latest	Clean outline icons
Frontend charts	Recharts / Chart.js	2.x	SHAP bar charts, urgency graphs
Backend runtime	Node.js + Express	20 LTS	Non-blocking I/O, easy streaming
Auth	jsonwebtoken + bcryptjs	latest	Stateless JWT, secure pw hashing
DB ORM	Prisma (or Knex)	5.x	Type-safe queries, migrations
Relational DB	PostgreSQL + pgvector	16	pgvector for embeddings (alternative to ChromaDB)
Vector store	ChromaDB	0.4	Zero-setup local vector DB
PDF parsing	pdf-parse (Node)	1.1	Extract text from uploaded PDFs
LLM / Embeddings	Google Gemini 1.5 Flash	API	Free tier, multimodal, 1M context
NLP sentiment	vader-sentiment (Node)	latest	Lexicon-based urgency scoring
ML model	Python + XGBoost + SHAP	2.x / 0.44	Attrition prediction + explainability
ML serving	Flask micro-service	3.x	Expose XGBoost as REST endpoint
Web search	SerpAPI / Google CSE	API	Live web search for AgentFlow
Streaming	Server-Sent Events (SSE)	native	Word-by-word LLM output

4. Database Design — Full Schema

All tables use UUID primary keys for portability. Timestamps use TIMESTAMPTZ. The vector store (ChromaDB or pgvector) is a separate collection/schema and is referenced by document ID.

4.1 Users & Sessions

Table: users

Column	Type	Constraints	Purpose
id	UUID	PRIMARY KEY DEFAULT gen_random_uuid()	Unique user identifier
email	VARCHAR(255)	UNIQUE NOT NULL	Login email
password_hash	VARCHAR(255)	NOT NULL	bcrypt hashed password
full_name	VARCHAR(255)	NOT NULL	Display name
role	ENUM	NOT NULL – see note	CLIENT EMPLOYEE IT_AGENT HR_MANAGER ADMIN
department	VARCHAR(100)	NULLABLE	Optional dept for employees
avatar_url	TEXT	NULLABLE	Profile picture URL
is_active	BOOLEAN	DEFAULT TRUE	Soft-delete / deactivation flag
created_at	TIMESTAMPTZ	DEFAULT NOW()	Account creation time
last_login_at	TIMESTAMPTZ	NULLABLE	Track engagement

Role ENUM values: CLIENT, EMPLOYEE, IT_AGENT, HR_MANAGER, ADMIN. Use a DB enum or a CHECK constraint.

Table: refresh_tokens

Column	Type	Constraints	Purpose
id	UUID	PRIMARY KEY	Token record ID
user_id	UUID	FK users(id) ON DELETE CASCADE	Owner
token_hash	VARCHAR(255)	NOT NULL	Hashed refresh token value
expires_at	TIMESTAMPTZ	NOT NULL	Expiry — typically 7 days
created_at	TIMESTAMPTZ	DEFAULT NOW()	Issued timestamp
revoked	BOOLEAN	DEFAULT FALSE	Allow explicit revocation

4.2 Tickets & Resolutions

Table: tickets

Column	Type	Constraints	Purpose
id	UUID	PRIMARY KEY	Ticket identifier
title	VARCHAR(255)	NOT NULL	Short problem summary
description	TEXT	NOT NULL	Full problem description (NLP input)
category	VARCHAR(80)	NOT NULL	Hardware Software Access Network Other
urgency_score	FLOAT	DEFAULT 0	VADER score 0.0 – 1.0
priority	ENUM	NOT NULL	LOW MEDIUM HIGH CRITICAL
status	ENUM	NOT NULL	OPEN IN_PROGRESS RESOLVED CLOSED
resolution_type	ENUM	NULLABLE	AUTO_RAG HUMAN ESCALATED
resolution_text	TEXT	NULLABLE	The resolution written by AI or human
rag_context	TEXT	NULLABLE	Chunks used by RAG (for audit)
created_by	UUID	FK users(id)	The client who raised the ticket
assigned_to	UUID	FK users(id) NULLABLE	IT agent assigned
created_at	TIMESTAMPTZ	DEFAULT NOW()	Submission time
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Last status change
resolved_at	TIMESTAMPTZ	NULLABLE	Resolution timestamp for SLA
sla_breach	BOOLEAN	DEFAULT FALSE	Flagged if resolution exceeded SLA window

Table: ticket_comments

Column	Type	Constraints	Purpose
id	UUID	PRIMARY KEY	Comment ID
ticket_id	UUID	FK tickets(id) ON DELETE CASCADE	Parent ticket
author_id	UUID	FK users(id)	Who wrote the comment
body	TEXT	NOT NULL	Comment content
is_internal	BOOLEAN	DEFAULT FALSE	If true, not visible to client
created_at	TIMESTAMPTZ	DEFAULT NOW()	Timestamp

4.3 Knowledge Base (RAG)

Table: documents

Column	Type	Constraints	Purpose
id	UUID	PRIMARY KEY	Document record ID
filename	VARCHAR(255)	NOT NULL	Original uploaded filename
file_path	TEXT	NOT NULL	Storage path on disk or S3 key
mime_type	VARCHAR(100)	NOT NULL	application/pdf text/plain
total_pages	INT	NULLABLE	Page count for PDFs
total_chunks	INT	DEFAULT 0	Number of embedded chunks
status	ENUM	NOT NULL	PENDING PROCESSING READY FAILED
uploaded_by	UUID	FK users(id)	Who uploaded
created_at	TIMESTAMPTZ	DEFAULT NOW()	Upload time

Table: document_chunks

Column	Type	Constraints	Purpose
id	UUID	PRIMARY KEY	Chunk record ID
document_id	UUID	FK documents(id) ON DELETE CASCADE	Parent document
chunk_index	INT	NOT NULL	Order within document
page_number	INT	NULLABLE	Source page — for citation
chunk_text	TEXT	NOT NULL	Raw text of the chunk (~500 tokens)
chroma_id	VARCHAR(255)	UNIQUE NOT NULL	ChromaDB / pgvector row ID
token_count	INT	DEFAULT 0	Approximate token count

Vector store note: Embeddings are NOT stored in PostgreSQL by default. The chroma_id column links to the ChromaDB collection nexusai_docs. If using pgvector, add an embedding vector(768) column to document_chunks and create a hnsw index for fast ANN search.

Table: rag_queries

Column	Type	Constraints	Purpose
id	UUID	PRIMARY KEY	Query log ID
user_id	UUID	FK users(id)	Who asked
question	TEXT	NOT NULL	User's natural language question
top_chunks	TEXT[]	NOT NULL	Array of chroma_ids used
answer	TEXT	NOT NULL	LLM-generated response
latency_ms	INT	NULLABLE	Response time for monitoring
feedback	SMALLINT	NULLABLE	Thumbs up=1, thumbs down=-1
created_at	TIMESTAMPTZ	DEFAULT NOW()	Query timestamp

4.4 Employee & HR Data

Table: employees

Column	Type	Constraints	Purpose
id	UUID	PRIMARY KEY	Employee ID
user_id	UUID	FK users(id) NULLABLE	Link to login if applicable
employee_code	VARCHAR(50)	UNIQUE NOT NULL	Internal HR code
full_name	VARCHAR(255)	NOT NULL	Employee name
department	VARCHAR(100)	NOT NULL	Engineering HR Sales etc.
job_title	VARCHAR(150)	NOT NULL	Current role
tenure_months	INT	NOT NULL	Months at company — key ML feature
age	INT	NOT NULL	Age in years
gender	VARCHAR(20)	NULLABLE	For diversity analytics
salary_band	VARCHAR(20)	NOT NULL	Junior Mid Senior Lead
salary_percentile	FLOAT	DEFAULT 50	Position in pay band 0–100
overtime_hours_month	FLOAT	DEFAULT 0	Avg monthly overtime hours
engagement_score	FLOAT	DEFAULT 50	Survey score 0–100
last_promotion_months	INT	DEFAULT 0	Months since last promotion
manager_id	UUID	FK employees(id) NULLABLE	Direct manager
work_location	ENUM	NOT NULL	ONSITE HYBRID REMOTE
is_active	BOOLEAN	DEFAULT TRUE	Active employment flag
hire_date	DATE	NOT NULL	Start date
created_at	TIMESTAMPTZ	DEFAULT NOW()	Record creation
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Last update

Table: attrition_predictions

Column	Type	Constraints	Purpose
id	UUID	PRIMARY KEY	Prediction record ID
employee_id	UUID	FK employees(id) ON DELETE CASCADE	Target employee
risk_score	FLOAT	NOT NULL	XGBoost probability 0.0–1.0
risk_label	ENUM	NOT NULL	LOW MEDIUM HIGH
shap_json	JSONB	NOT NULL	SHAP values per feature as JSON object
model_version	VARCHAR(20)	NOT NULL	Model version tag for auditability
predicted_at	TIMESTAMPTZ	DEFAULT NOW()	When prediction was run

SHAP JSON shape: { "overtime_hours_month": 0.23, "last_promotion_months": 0.18, "engagement_score": -0.15, "tenure_months": -0.12 } — positive values push risk up, negative push risk

down.

4.5 Agent Sessions

Table: agent_sessions

Column	Type	Constraints	Purpose
id	UUID	PRIMARY KEY	Session ID
user_id	UUID	FK users(id)	Who triggered the search
query	TEXT	NOT NULL	Original user question
react_trace	JSONB	NOT NULL	Full Thought/Action/Observation loop as JSON array
final_report	TEXT	NOT NULL	Synthesised markdown report
source_urls	TEXT[]	DEFAULT '{}'	Cited source URLs
loop_count	INT	DEFAULT 0	Number of ReAct iterations
latency_ms	INT	NULLABLE	Total response time
created_at	TIMESTAMPTZ	DEFAULT NOW()	Session timestamp

5. Auth Gate — Role-Based Access Control

The Auth Gate is the **entry point for every request**. It uses stateless JWT access tokens (15-minute expiry) plus refresh tokens (7-day expiry) stored in the database. On each request, the Express middleware decodes the JWT, loads the role, and gates the downstream route.

Step-by-step flow:

Step	Action	Component	Input / Output
1	User submits email + password	POST /api/auth/login	IN: { email, password }
2	Server fetches user by email from DB	users table query	OUT: user record or 401
3	Compare password against hash	bcryptjs.compare()	OUT: boolean match
4	Generate access token (15 min)	jsonwebtoken.sign({ userId, role })	OUT: accessToken JWT
5	Generate refresh token, store hash in DB	refresh_tokens table insert	OUT: refreshToken
6	Return both tokens to client	HTTP 200 + Set-Cookie or JSON	Client stores in memory / httpOnly cookie
7	Client attaches access token to every request	Authorization: Bearer	All subsequent API calls
8	Middleware decodes + validates token	jwt.verify() middleware	Attaches req.user = { userId, role }
9	Role scope guard checks route permission	roleGuard(REQUIRED_ROLE)	403 if insufficient role
10	Route handler executes	Module-specific logic	Response returned to client

Role permission matrix:

Route / Feature	CLIENT	EMPLOYEE / IT_AGENT	HR_MANAGER	ADMIN
Submit ticket	YES	NO	NO	YES
View own tickets	YES	NO	NO	YES
View all ticket queue	NO	YES	YES	YES
Resolve / close ticket	NO	YES	YES	YES

Upload documents (DocuMind)	NO	YES	YES	YES
Query documents (DocuMind)	YES	YES	YES	YES
View SentinelHR dashboard	NO	NO	YES	YES
Run attrition prediction	NO	NO	YES	YES
Use AgentFlow	NO	YES	YES	YES
Manage users (Admin)	NO	NO	NO	YES

6. TicketConsole — AI-Powered Incident Handler

TicketConsole runs a **dual-model NLP pipeline** on every incoming ticket. VADER scores the emotional urgency of the text. A rule-based BERT-lite classifier categorises the issue. A RAG lookup attempts automatic resolution before handing off to the human queue.

Full pipeline flow:

Step	Action	Component	Input / Output
1	Client submits ticket form	POST /api/tickets	IN: { title, description, category }
2	VADER sentiment analysis	vader-sentiment library	OUT: compound score -1 to +1 → urgency 0–100%
3	Derive priority from urgency	Server logic	score>0.7→CRITICAL, >0.5→HIGH, >0.3→MEDIUM, else LOW
4	BERT-lite category classification	Keyword rules + intent matching	OUT: Hardware Software Access Network Other
5	Save ticket to DB with scores	tickets INSERT	Status=OPEN, urgency_score, priority, category stored
6	Search FAQ / knowledge base via RAG	ChromaDB cosine similarity	IN: ticket description embedding, OUT: top 3 chunks
7	If best match score > 0.82	Confidence threshold check	Proceed to auto-resolution branch
8	Build augmented prompt with FAQ context	Gemini 1.5 Flash API call	IN: ticket + context, OUT: resolution text
9	Save resolution, set status=RESOLVED	tickets UPDATE	resolution_type=AUTO_RAG, resolved_at=NOW()
10	Stream resolution to client instantly	SSE response	Client sees answer in real time
11	If no match — set status=OPEN, add to queue	No LLM call	Employees see it in queue sorted by urgency
12	Employee views queue, picks ticket	GET /api/tickets?status=OPEN	Sorted by priority desc, created_at asc
13	Employee writes resolution, submits	PATCH /api/tickets/:id	IN: { resolution_text }, status → RESOLVED
14	SLA breach checked on resolution	Server: resolved_at - created_at	If > SLA window, sla_breach=TRUE

VADER urgency scoring logic:

```
// vader-sentiment returns compound score: -1 (very negative) to +1 (very positive)
```

```
// For urgency we invert: very negative text = very urgent
const vader = new SentimentIntensityAnalyzer();
const scores = vader.polarity_scores(description);
const urgency = Math.round((1 - scores.compound) / 2 * 100);
// Boost urgency if ALL_CAPS words detected (shouting = urgency signal)
const capsRatio = (description.match(/[A-Z]{3,}/g) || []).length / words.length;
const finalUrgency = Math.min(100, urgency + capsRatio * 20);
```

7. DocuMind — Enterprise RAG Document Q&A;

DocuMind implements a full **Retrieval-Augmented Generation (RAG)** pipeline. PDFs are parsed, chunked, embedded into a vector store, and made queryable via semantic similarity search. The LLM is strictly constrained to answer only from retrieved context — preventing hallucinations.

RAG pipeline — upload phase:

Step	Action	Component	Input / Output
1	User uploads PDF via browser	POST /api/documents/upload (multipart)	IN: file binary, filename
2	Save file to disk / S3	multer middleware	OUT: file_path stored in documents table
3	Insert documents record, status=PENDING	documents INSERT	OUT: document_id UUID
4	Extract raw text from PDF	pdf-parse Node library	OUT: full text string + page map
5	Split text into ~500-token chunks with 50-token overlap	Custom chunker	OUT: array of { chunk_text, page_number, chunk_index }
6	Embed each chunk via Gemini API	models/embedding-001 batchEmbedContents	OUT: 768-dim float vector per chunk
7	Store chunk metadata in document_chunks table	document_chunks INSERT	chroma_id = UUID assigned
8	Store vectors in ChromaDB collection nexusai_docs	chromadb.Collection.add()	IN: ids, embeddings, metadatas, documents
9	Update document status to READY	documents UPDATE	total_chunks count updated

RAG pipeline — query phase:

Step	Action	Component	Input / Output
1	User types question in chat box	POST /api/documents/query	IN: { question, document_id? }
2	Embed the question	Gemini embedding API	OUT: 768-dim query vector
3	Cosine similarity search in ChromaDB	collection.query(n_results=5)	OUT: top 5 chunks + distance scores
4	Filter chunks with distance > 0.75 threshold	Server-side filtering	Discard low-relevance chunks

5	Build augmented prompt	Prompt engineering	System: 'Answer ONLY from context. Cite page numbers.'
6	Call Gemini 1.5 Flash with prompt + context	Gemini generateContent()	OUT: answer text with citations
7	Stream response to frontend via SSE	res.write() chunks	Client renders word-by-word
8	Log query + answer to rag_queries table	rag_queries INSERT	For audit + feedback loop
9	User rates response (thumbs up/down)	PATCH /api/rag-queries/:id/feedback	Stored for future fine-tuning signal

Chunking strategy — technical detail:

```
function chunkText(fullText, chunkSize=500, overlap=50) {
  const words = fullText.split(/\s+/);
  const chunks = [];
  let start = 0;
  while (start < words.length) {
    const end = Math.min(start + chunkSize, words.length);
    chunks.push(words.slice(start, end).join(' '));
    start += chunkSize - overlap; // 50-token overlap preserves context at boundaries
  }
  return chunks;
}
```


8. SentinelHR — XGBoost Attrition Predictor

SentinelHR uses a pre-trained **XGBoost gradient boosting model** to predict the probability of an employee leaving within the next 6 months. **SHAP values** decompose each prediction into per-feature contributions, giving HR managers actionable insight rather than a black-box score.

Model training (offline, one-time setup):

```
# Train on IBM HR Analytics dataset (Kaggle) or synthetic company data
import xgboost as xgb, shap, joblib, pandas as pd

FEATURES = ['age', 'tenure_months', 'overtime_hours_month', 'engagement_score',
'salary_percentile', 'last_promotion_months', 'department_enc',
'work_location_enc', 'job_level_enc']

model = xgb.XGBClassifier(n_estimators=200, max_depth=5, learning_rate=0.05,
use_label_encoder=False, eval_metric='logloss')
model.fit(X_train, y_train)
joblib.dump(model, 'attrition_model_v1.pkl')

# SHAP explainer – save separately for fast inference
explainer = shap.TreeExplainer(model)
joblib.dump(explainer, 'shap_explainer_v1.pkl')
```

Prediction flow (live request):

Step	Action	Component	Input / Output
1	HR Manager selects employee from list	GET /api/hr/employees	IN: employee_id query param
2	Node server proxies to Python Flask service	POST http://localhost:5001/predict	IN: { employee_id }
3	Flask loads employee record from DB	SELECT * FROM employees WHERE id=?	OUT: feature dict
4	Encode categorical fields	LabelEncoder (department, work_location)	OUT: numeric feature vector
5	XGBoost model.predict_proba()	Loaded from attrition_model_v1.pkl	OUT: [p_stay, p_leave] float pair
6	SHAP explainer.shap_values()	Loaded from shap_explainer_v1.pkl	OUT: shap_values array per feature
7	Build response JSON	Python dict serialisation	OUT: { risk_score, risk_label, shap_values: {feature: value} }

8	Save to attrition_predictions table	INSERT via SQLAlchemy	model_version, predicted_at stored
9	Node server returns JSON to frontend	HTTP 200 from Flask → forwarded	Risk gauge + SHAP bar chart data
10	Frontend renders risk gauge + SHAP chart	Recharts HorizontalBar	Positive SHAP = coral bars, Negative = teal bars

SHAP response structure:

```
{
  "employee_id": "uuid-...",
  "risk_score": 0.74,
  "risk_label": "HIGH",
  "shap_values": {
    "overtime_hours_month": +0.231, // pushing risk UP
    "last_promotion_months": +0.187,
    "engagement_score": -0.152, // pulling risk DOWN
    "tenure_months": -0.118,
    "salary_percentile": +0.097
  },
  "model_version": "v1.0",
  "predicted_at": "2025-06-03T14:22:11Z"
}
```

9. AgentFlow — ReAct Web Research Agent

AgentFlow implements the **ReAct (Reason + Act) framework** using Google Gemini 1.5 Flash with tool use enabled. Unlike a simple LLM call, the agent plans its own search queries, executes live web searches, reads results, and decides whether to search again — up to 3 iterations — before synthesising a grounded, citation-backed report.

ReAct loop flow:

Step	Action	Component	Input / Output
1	User submits research query	POST /api/agent/search	IN: { query }
2	Server creates agent session record	agent_sessions INSERT (status=RUNNING)	OUT: session_id
3	Initialise Gemini with ReAct system prompt	Gemini Chat session with tools	System prompt defines Thought/Action/Observation format
4	THOUGHT — Gemini plans first search query	Gemini generateContent()	OUT: { thought: string, action: { tool: 'web_search', query: string } }
5	ACTION — Server calls SerpAPI with Gemini's query	SerpAPI GET /search?q=...	OUT: top 5 organic results { title, snippet, url }
6	OBSERVATION — Pass results back to Gemini	Next turn in chat history	IN: serialised search results
7	THOUGHT — Gemini decides: enough data or search again	Gemini generateContent()	If insufficient: loops back to step 4
8	Loop repeats up to 3 times max	Loop counter check	Prevents infinite loops and API cost overrun
9	SYNTHESIS — Gemini writes final report	Final generateContent() call	No tool call in response = synthesis mode
10	Clean output: strip markdown symbols, format tables	Server post-processor	Replace ** with UPPERCASE headings
11	Stream report to frontend via SSE	res.write() per sentence	Frontend renders ReAct trace + report side by side
12	Save full trace + report to agent_sessions	agent_sessions UPDATE	react_trace as JSONB, source_urls array

System prompt (ReAct instruction):

You are a factual research agent. Always follow this format strictly:

THOUGHT: [Reason about what information you need and why]
ACTION: web_search query: [your exact search query]
OBSERVATION: [I will provide search results here]
... (repeat THOUGHT/ACTION/OBSERVATION up to 3 times)
FINAL REPORT:
[Write a structured report using UPPERCASE section headings.
Include a SOURCES section at the end with numbered URLs.
Do NOT use asterisks (*) or markdown bold (**). No fabricated facts.]

ReAct trace JSON structure (stored in agent_sessions.react_trace):

```
[
  { "step": 1, "type": "thought", "content": "I need to find recent benchmarks..." },
  { "step": 2, "type": "action", "tool": "web_search", "query": "LangChain vs LlamaIndex 2025 benchmark" },
  { "step": 3, "type": "observation", "results": [{ "title": "...", "snippet": "...", "url": "..."} ] },
  { "step": 4, "type": "thought", "content": "I need more details on memory management..." },
  { "step": 5, "type": "action", "tool": "web_search", "query": "LlamaIndex memory architecture 2025" },
  { "step": 6, "type": "observation", "results": [...] },
  { "step": 7, "type": "synthesis", "report": "COMPARISON: LANGCHAIN VS LLAMAINDEX..." }
]
```

10. API Endpoints Reference

Method + Path	Auth Role	Request Body	Response
POST /api/auth/register	Public	{ email, password, fullName, role }	{ accessToken, refreshToken, user }
POST /api/auth/login	Public	{ email, password }	{ accessToken, refreshToken, user }
POST /api/auth/refresh	Public	{ refreshToken }	{ accessToken }
POST /api/auth/logout	Any auth	{ refreshToken }	204 No Content
GET /api/tickets	EMPLOYEE+	Query: status, priority, category	Array of ticket objects
POST /api/tickets	CLIENT	{ title, description, category }	{ ticket, resolution? }
GET /api/tickets/:id	CLIENT (own) / EMPLOYEE+	—	Full ticket object with comments
PATCH /api/tickets/:id	EMPLOYEE+	{ resolution_text, status }	Updated ticket
POST /api/documents/upload	EMPLOYEE+	multipart/form-data: file	{ document_id, status }
GET /api/documents	Any auth	—	List of READY documents
POST /api/documents/query	Any auth	{ question, document_id? }	SSE stream: answer text
GET /api/hr/employees	HR_MANAGER+	Query: department, risk_label	Array of employee summaries
GET /api/hr/employees/:id/predict	HR_MANAGER+	—	{ risk_score, risk_label, shap_values }
POST /api/agent/search	EMPLOYEE+	{ query }	SSE stream: ReAct trace + final report
GET /api/agent/sessions	EMPLOYEE+	—	Past agent sessions for user
GET /api/admin/users	ADMIN	—	All users
PATCH /api/admin/users/:id	ADMIN	{ role, is_active }	Updated user

11. Frontend Structure & UX Flow

Folder structure:

```
src/
components/
auth/ LoginForm.jsx, RegisterForm.jsx, DemoRolePicker.jsx
layout/ Sidebar.jsx, Header.jsx, RoleGate.jsx
tickets/ TicketForm.jsx, TicketQueue.jsx, TicketCard.jsx
docmind/ UploadZone.jsx, ChatWindow.jsx, SourceCitation.jsx
sentinelhr/ EmployeeList.jsx, RiskGauge.jsx, ShapChart.jsx
agentflow/ SearchBar.jsx, ReactTracePanel.jsx, ReportViewer.jsx
pages/
Dashboard.jsx, Tickets.jsx, DocuMind.jsx, SentinelHR.jsx,
AgentFlow.jsx, Admin.jsx
hooks/
useAuth.js, useSSE.js, useTickets.js
lib/
api.js (axios instance with JWT interceptor)
roleConfig.js (maps role → allowed routes)
App.jsx (React Router v6 with ProtectedRoute wrapper)
main.jsx
```

Key UX decisions:

- **Demo role picker on login page** — three buttons (Client / IT Agent / HR Manager) pre-fill credentials and log in instantly. Essential for designathon judging.
- **Sidebar collapses unused modules** — modules the user's role cannot access are greyed out with a lock icon, not hidden, so judges can see the full feature set.
- **SSE streaming for DocuMind and AgentFlow** — use the EventSource API. The custom useSSE() hook handles connection, message parsing, and cleanup.
- **Ticket queue auto-refreshes** — WebSocket or polling every 15 seconds so agents see new tickets without refreshing.
- **SHAP chart uses Recharts HorizontalBar** — positive SHAP values (risk drivers) in coral/red, negative (protective factors) in teal. Sorted by absolute value descending.
- **ReAct trace is collapsible** — show the final report by default, let users expand to see Thought/Action/Observation steps. Impressive to demo but not overwhelming.
- **Toast notifications** — ticket submitted, document processed, prediction complete. Use react-hot-toast.

12. Deployment Guide

Local development setup:

```
# 1. Clone and install
git clone https://github.com/yourname/nexusai && cd nexusai
npm install # root: installs backend deps
cd client && npm install # installs frontend deps

# 2. Environment variables (.env in project root)
DATABASE_URL=postgresql://user:pass@localhost:5432/nexusai
JWT_SECRET=your-256-bit-secret
JWT_REFRESH_SECRET=another-256-bit-secret
GEMINI_API_KEY=AIZA...
SERPAPI_KEY=your-serpapi-key
CHROMA_HOST=localhost
CHROMA_PORT=8000
PYTHON_ML_SERVICE=http://localhost:5001

# 3. Start ChromaDB (Docker)
docker run -p 8000:8000 chromadb/chroma

# 4. Run database migrations
npx prisma migrate dev --name init

# 5. Start Python ML service
cd ml-service && pip install flask xgboost shap joblib
python app.py

# 6. Start backend + frontend (two terminals)
npm run dev # Express on :3001
cd client && npm run dev # Vite on :5173
```

Production deployment (Render / Railway):

Service	Platform	Notes
React SPA	Vercel / Netlify	Static build, set VITE_API_URL env var
Node.js API	Render / Railway	Free tier works; set all .env vars in dashboard
PostgreSQL	Supabase / Neon	Free tier, copy connection string to DATABASE_URL
ChromaDB	Render Docker service	Use persistent disk mount for data volume
Python ML service	Render / Fly.io	Deploy as separate service, set PYTHON_ML_SERVICE_URL

13. Adaptability for Real Companies

NexusAI is architected so that a real company can adopt it with minimal changes. The following table maps each module to its enterprise adaptation path.

Module	What to swap / configure	Integration points
Auth Gate	Replace local users table with Azure AD / Okta SSO via OAuth2	Passport.js strategy for SAML/OIDC; roles map from AD groups
TicketConsole	Import existing tickets from Jira / ServiceNow via REST API	Webhook receiver for new tickets; outbound webhook on resolution
DocuMind RAG	Connect to SharePoint / Confluence instead of file upload	MCP connector or REST API to fetch pages; same chunking pipeline
SentinelHR	Load employees from HRIS (Workday / BambooHR)	HR API data sync job; retrain XGBoost on company's own attrition history
AgentFlow	Swap SerpAPI for internal Intranet search or Bing API	Tool definition in Gemini is configurable; add custom internal search tool
Database	Migrate SQLite (dev) → PostgreSQL (prod) — Prisma handles this	Run: prisma migrate deploy on prod environment
LLM	Swap Gemini for Azure OpenAI or Anthropic Claude via env var	Abstract LLM calls behind an <code>IlmClient</code> interface; swap provider in config
Branding	Company name, logo, and color theme via Tailwind config and env	theme.js with primary/secondary colors; logo in public/

The designathon pitch in one line:

"NexusAI is a production-ready enterprise AI operations platform that uses RAG for knowledge retrieval, supervised ML for predictive HR analytics, and autonomous ReAct agents for research — all behind a secure, role-based dashboard that any company can adopt in a week."

End of Technical Specification

NexusAI Platform — v1.0 — Designathon 2025